Slide-1

1.Define Compiler

Ans: A compiler act as a translator, transforming human-oriented programming language (high level language) into computer-oriented machine language(low level language).

2.Difference between compiler and interpreter.

Compiler	Interpreter
Scans the entire program and translate it as a whole into machine code	Translate program one statement at a time.
Fast execution	Slow execution
Memory require is more	Memory require is less
Intermediate object code is generated	Not intermediate object code is generated
Errors are displayed after entire program is checked	Errors are displayed for every instruction interpreted

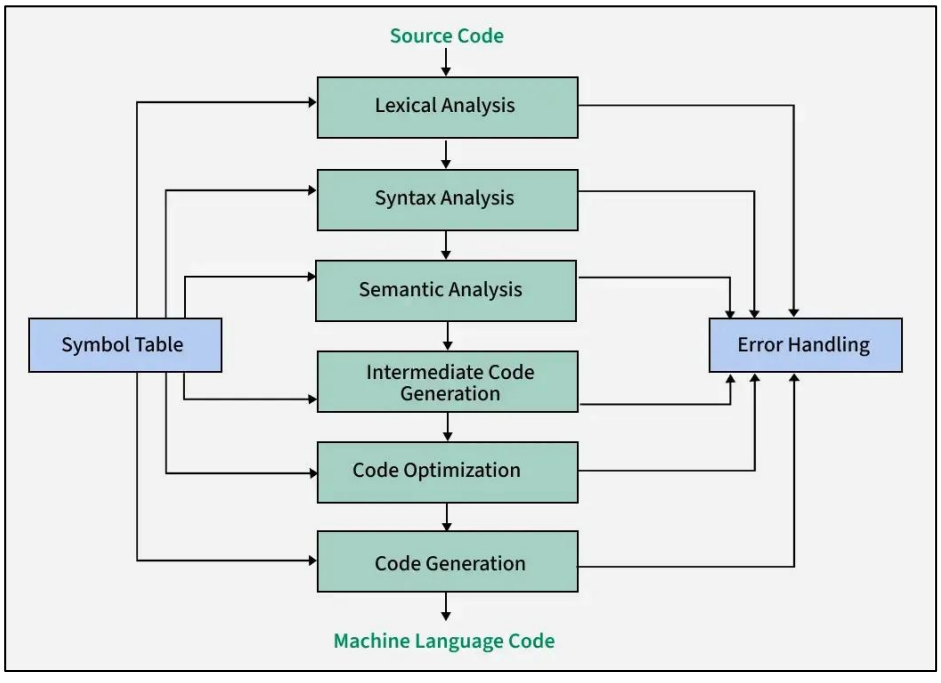
3.Program execution steps.

- User writes a program in C language (high -level language.)
- The C compiler, compiles the program and translates it to assembly program (low -level language).
- An assembler then translates the assembly program into machine code object
- A linker tool is used to link all the parts of the program together for execution
- Executable machine code.
- A loader loads all of them into memory and then the program is executed.

4.Analysis and Synthesis phase of compiler.

- Optimizes the intermediate code
- Allocates memory and registers
- Generates target machine instructions
- Produces final executable machine code

5.Dscribe the phase of compiler.



Slide-2

6.The token classification (classify token into 6 categories + count total tokens present in the given program).

Ans:

Token Classification in Compiler Design

In Compiler Design, a **token** is the smallest meaningful unit of a program.
During lexical analysis, the compiler breaks the program into tokens.

Tokens are mainly classified into **6 categories**:

---

1. Keywords

- Keywords are reserved words.
- They have fixed meaning in programming language.
- We cannot use them as variable names.
- Example (in C language):
    - int
    - float
    - if
    - else
    - return
    - while

Example:

int x;

Here, int is a keyword.

---

2. Identifiers

- Identifiers are names given to variables, functions, arrays, etc.
- They are user-defined.
- Must start with a letter or underscore (_).
- Cannot be a keyword.

Example:

int totalMarks;

Here, totalMarks is an identifier.

---

3. Constants (Literals)

- Fixed values that do not change.
- Can be integer, float, character, or string.

Examples:

- 10
- 3.14
- 'A'
- "Hello"

Example:

int x = 5;

Here, 5 is a constant.

---

4. Operators

- Operators perform operations on variables and values.
- Types:
    - Arithmetic: +, -, *, /
    - Relational: >, <, ==
    - Logical: &&, ||
    - Assignment: =

Example:

x = a + b;

Here, =, + are operators.

---

5. Special Symbols (Separators / Punctuators)

- Used to separate statements and symbols.
- Examples:
    - ;
    - ,
    - ()
    - {}
    - []

Example:

int x;

Here, ; is a special symbol.

---

6. Comments

- Used to explain code.
- Ignored by compiler.
- Example:

- // This is a comment
- /* multi-line comment */

Example Program and Token Count

Consider this C program:

```
int main() {
  int a = 5;
  int b = 10;
  int sum = a + b;
  return sum;
}
```

Now let's count tokens one by one.

Line 1:

int → Keyword (1)
main → Identifier (2)
(→ Special Symbol (3)
) → Special Symbol (4)
{ → Special Symbol (5)

Line 2:

int → Keyword (6)
a → Identifier (7)
= → Operator (8)
5 → Constant (9)
; → Special Symbol (10)

Line 3:

int → Keyword (11)
b → Identifier (12)
= → Operator (13)
10 → Constant (14)
; → Special Symbol (15)

Line 4:

int → Keyword (16)
sum → Identifier (17)
= → Operator (18)
a → Identifier (19)
+ → Operator (20)
b → Identifier (21)
; → Special Symbol (22)

Line 5:

return → Keyword (23)
sum → Identifier (24)
; → Special Symbol (25)

Line 6:

} → Special Symbol (26)

☑ **Total Tokens = 26**

Conclusion

Tokens are the basic building blocks of a program.
They are divided into 6 main categories:

1. Keywords
2. Identifiers
3. Constants
4. Operators
5. Special Symbols
6. Comments

The lexical analyzer identifies these tokens before syntax analysis starts.

7.Errors and Error recovery.

Ans: Lexical Error

- lexical error occurs when a sequence of characters does not match the pattern of any token.
- It typically happens during the execution of a program.
- Types of errors:
    - Exceeding length of identifier or numeric constants.
    - Appearance of illegal characters
    - Unmatched string
    - Spelling Error

Error Recovery Techniques

1.Panic Mode Recovery
2.Transpose of two adjacent characters.
3.Insert a missing character
4.Delete an unknown or extra character
5.Replace a character with another.

Slide-3

8.what is grammar?

Ans: Grammar is a formal system that defines a set of rules for generating valid strings within a language.

- Grammar is Composed of two basic elements:
  - Terminal Symbols:Terminal symbols are those that are the components of the sentences generated using grammar and are represented using small case letters like a, b, c, etc.
  - Non-terminal Symbols : Non-terminal symbols are those symbols that take part in the generation of the sentence but are not the component of the sentence.These symbols are represented using a capital letters like A, B, C, etc.

9.Explain the four types of grammars in the Chomsky Hierarchy with examples. (type0,type1,type2,type3 with example)

Ans: Four Types of Grammars in the Chomsky Hierarchy

The **Noam Chomsky** proposed a classification system for formal grammars called the **Chomsky Hierarchy**.
It divides grammars into four types based on their power and restrictions.

The four types are:
Type 0, Type 1, Type 2, and Type 3.

---

1. Type 0 Grammar (Unrestricted Grammar)

- It is the most powerful grammar.
- Also called **Unrestricted Grammar**.
- There are no restrictions on production rules.
- The rule format is:
  α → β
  (Where α and β can be any combination of terminals and non-terminals.)
- It is recognized by a **Turing Machine**.
- It can generate all languages that are recursively enumerable.

Example:

S → aSb
S → ab

This grammar can generate:
ab, aabb, aaabbb, etc.

---

2. Type 1 Grammar (Context-Sensitive Grammar)

- Also called **Context-Sensitive Grammar (CSG)**.

- Production rules depend on context.
- Rule format:
  αAβ → αγβ
  (A can be replaced depending on surrounding symbols.)
- Length of left side ≤ length of right side.
- Recognized by a Linear Bounded Automaton.
- More powerful than Type 2 but less than Type 0.

Example:

S → aSBC
S → abc
CB → BC
aB → ab
bB → bb

This grammar generates strings like:
abc, aabbcc, aaabbbccc

---

3. Type 2 Grammar (Context-Free Grammar)

- Also called **Context-Free Grammar (CFG)**.
- Most commonly used in programming languages.
- Production rule format:
  A → γ
  (Left side must have only one non-terminal.)
- Recognized by a Pushdown Automaton (PDA).
- Used in syntax analysis (parsing).

Example:

S → aSb
S → ε

This grammar generates:
ε, ab, aabb, aaabbb

---

4. Type 3 Grammar (Regular Grammar)

- Also called **Regular Grammar**.
- Least powerful grammar.
- Production rule format:
  - A → aB
  - A → a
- Recognized by Finite Automata.
- Used in lexical analysis of compilers.

Example:

S → aA
A → bS
S → a

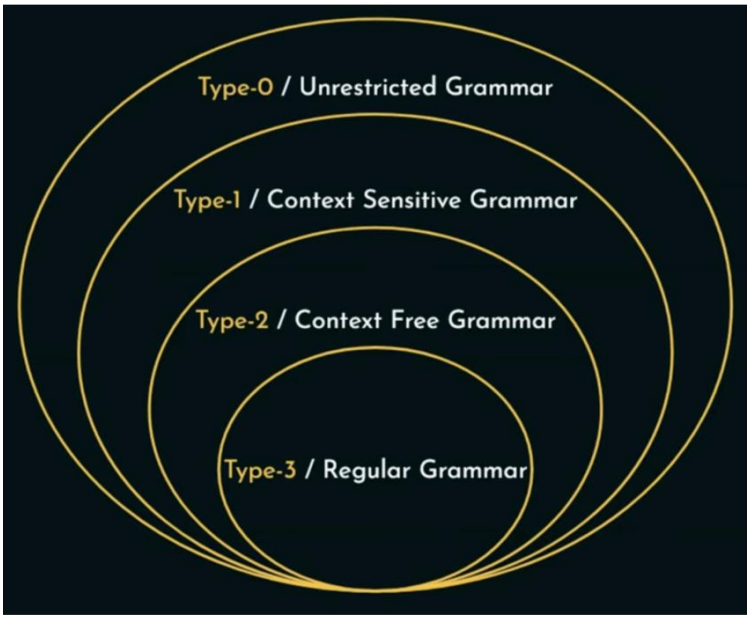
This generates strings like:
a, aba, ababa

Conclusion

The power order is:

Type 0 > Type 1 > Type 2 > Type 3

- Type 3 is simplest (used in lexical analysis).
- Type 2 is used for programming language syntax.
- Type 1 handles more complex structures.
- Type 0 is the most powerful and general.



Grammar

10.Regular Expression + Example

Ans: A **Regular Expression** is a pattern used to describe a set of strings.
It is mainly used in lexical analysis in compiler design.
With the help of Regular Expression, we can define tokens like identifiers, numbers, keywords, etc.

Regular Expressions are used to represent **Regular Languages**.

Example 1:

•Write the regular expression for the language accepting all combinations of a's, over the set ∑ = {a}

Solution:

•All combinations of a's means a may be zero, single, double and so on. If a is appearing zero times, that means a null string. That is we expect the set of {ε, a, aa, aaa, ...}. So we give a regular expression for this as:

R=a*

That is Kleen closure of a.

Example 2:

•Write the regular expression for the language accepting all combinations of a's except the null string, over the set ∑ = {a}

Solution:

The regular expression has to be built for the language

L={a,aa,aaa,....}

This set indicates that there is no null string. So we can denote regular expression as:

R = a+

Example 3:

•Write the regular expression for the language accepting all the string containing any number of a's and b's.

Solution:

•The regular expression will be:

r.e.=(a+b)*

This will give the set as L = {ε, a, aa, b, bb, ab, ba, aba, bab, ...}, any combination of a and b.

The (a + b)* shows any combination with a and b even a null string.

Example 4:

•Write the regular expression for the language accepting all the string which are starting with 1 and ending with 0, over ∑ = {0, 1}.

Solution:

•In a regular expression, the first symbol should be 1, and the last symbol should be 0. The r.e. is as follows:

R.E.=1(0+1)*0

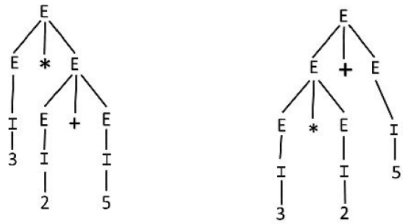
11.what is ambiguous grammar

Ans: A grammar is said to be ambiguous if there exists more than one leftmost derivation or more than one rightmost derivation or more than one parse tree for the given input string If the grammar is not ambiguous, then it is called unambiguous

12. Check whether grammar is ambiguous

Examples of Ambiguous Grammar

- Example 1:
- Let us consider a grammar G with the production rule
- E → I
- E → E + E
- E → E * E
- E → (E)
- I → ε | 0 | 1 | 2 | ... | 9



For the string "3 * 2 + 5", the above grammar Can generate two parse trees

Q: Check whether the grammar is ambiguous or not.

Examples of Ambiguous Grammar (Cont.)

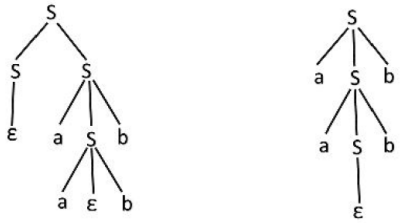
Example 2:

- Check whether the given grammar G is ambiguous or not.

S → aSb | SS

S → ε

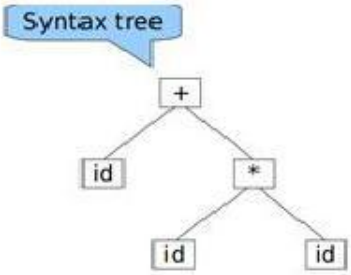
For the string "aabb" the above grammar can generate two parse trees



Since there are two parse trees for a single string "aabb", the grammar G is ambiguous.

Q: Check whether the grammar is ambiguous or not.

13.Difference between parse tree and syntax tree.

Syntax Tree	Parse Tree
interior nodes are "operators", leaves are operands	interior nodes are non-terminals, leaves are terminals
when representing a program in a tree structure usually use a syntax tree	rarely constructed as a data structure
Represents the abstract syntax of a program (the semantics)	Represents the concrete syntax of a program
Contain only meaningful information.	Contain unusable information also.
Syntax tree examples Grammar: E → E * E | E + E | id Program: a + b * c 	Parse tree examples Grammar: E → E * E | E + E | id Program: a + b * c